Capstone Project - Deep Learning Study

Divij Khaitan

Supervisor: Subhashis Banerjee

February 6, 2025

Contents

1 Introduction **1**
1.1 Related work 1
1.2 Difference from existing work 2

2 Methods **3**
2.1 Class-Wise Frequencies of Significant Neuron-Weight Interactions 3
2.2 KL Divergence Between Classes 4
2.3 Modelling Activations using a Multivariate Gaussian 4
2.4 Memorisation and Significance 5
2.5 Adversarial Attacks 5
2.6 Manifold Entaglement 5

3 Results **5**
3.1 Sparsity of Activations 6
3.2 Divergences of Neuron Weight Interactions 8
3.3 Adversarial Attacks and Probability Distributions 14
3.4 Manifold Entanglement 15

4 Conclusions and Future Work **16**

A Details for Modified Alexnet **19**

B Appendix B: PGD Attack Details **19**

1 Introduction

The goal of this project is to introduce a new framework for the analysis of the robustness properties of deep neural networks by analysing the patterns of neuron activations. The goals of this framework are

1. Defining 'training distributions' with regards to a trained network as opposed to a training dataset
2. Provide a diagnostic tool to detect when a network has overfit with polynomial overhead over the training process

1.1 Related work

Generalisation/Implicit Regularisation: Zhang et. al. showed as early as 2017 that even AlexNet has enough capacity to fit all of imagenet with completely random labels perfectly when given enough time [30]. This suggests that although networks have extremely high 'capacity', they do not use all of it in practice and that contrary to statistical wisdom, they are not overfitting as a result of overparameterisation. Arora et. al. showed that overparameterisation provides a speed up to the optimisation of neural networks and provided bounds for the generalisation error of two layer ReLU networks [1]. Soltanolkotabi and Oymak showed that networks that polynomially overparameterised in the size of the input are resistant to label corrpution, with the final parameters being close to their initialised values [14]. A major hypothesis in this area is that stochastic gradient descent (SGD) as a training method introduces an 'implicit regularisation' into the training of a deep network. Norm minimisation was one explanation for this behaviour, first put forth by [20]. Several notions of complexity have been put forth, including VC dimension and Rademacher complexity, see [19] for a more granular view of these. Examples where all matrix norms diverge were constructed in [23], indicating that the relationship between weight decay and implicit regularisation requires more inspection. This relationship has also been questioned by [2], which shows that for the case of matrix factorisation, matrix norms are insufficient to describe the action of gradient-based optimisation. This paper also shows that adding depth to matrix factorisation encourages low rank solutions, which improve the accuracy of the reconstruction. [6] formally showed that deep networks learn 'incrementally', and while shallow networks can exhibit similar behaviour they do so for a small set of initial conditions. [3] showed that SGD behaves differently on noise as opposed to actual datasets, and that dropout works better than noise injection for hindering memorisation while promoting generalisation

Out of Distribution Detection: When considering open world deployment of networks, it simply isn't possible to train them on exhaustive partition of every input they can expect. Thus, it becomes important to develop mechanisms that allow users to detect when an input is outside the training distribution of a model. The first major work that tackled this problem with some effectiveness

was [9], which uses $-\max_i \operatorname{softmax}(f(x)_i)$ as a means of detecting if a particular input is outside the training distribution. This is analogous to treating the magnitude of the softmax output as a confidence measure, which isn't entirely accurate as pointed out by the authors. However, this has proven to be a baseline which is difficult to beat non-trivially. [10] describes a method for fine-tuning trained models to detect OOD inputs using unlabelled data and the KL divergence from a uniform distribution. ODIN [15] has empirically shown excellent performance on OOD detection, but requires tuning hyperparameters beforehand. [13] Uses a similar idea of creating a probability distribution over feature spaces and uses the minimum mahalanobis distance as a classifier, and also use it to show that tuning their classifier on certain adversarial attacks adds robustness to other adversarial attacks. The authors assume a multivariate gaussian distribution on the class-conditional softmax probability vectors, and for a vector output the class for which the probability vector has the smallest mahalanobis distance. [25] Uses the gram matrices of the feature vectors and computes layer-wise deviations of an input from the normal values of the class.

Activation Patterns of Neural Networks: There has been some past work on analysing the patterns of activation of neural networks. On the side of interpretability, SUMMIT [11] and NAP [4] both attempt to identify important neurons in an attempt to interpret the features associated with different classes. [28] fits a mixture of gaussians to each neuron independently in selected layers with the explicit goal of attributing predictions to neurons. They use at most 2 gaussians Similar ideas have been used for the detection of rare classes [22] as well as the detection of out-of-distribution inputs [21]. To our knowledge, this is the first work that imposes a notion of significance on the activations.

Adversarial Robustness: Szegdy et. al. discovered [29] in 2014 that classifiers on vision models are susceptible to extremely small changes (in some L^p space) that can cause a misclassfification. Since then, a considerable amount of work has gone into attempting to explain their existence ([26], [27]) and the development of attacks (PGD [16], FGSM[7]) and defenses to those attacks, either by changing the loss function or by modifying the training algorithm in some way.

Manifold Hypothesis: This is an idea that pre-dates deep learning, that most real-world datasets in high dimensions have an underlying low dimensional structure. In the context of machine learning, it has primarily been studied from the perspective of dimensionality reduction, reducing a set of points in a high dimensional space to one in a low dimensiona space which is equivalent to it. [5] develops an algorithm to test the hypothesis, and [18] discusses the sample complexity of the problem. See [17] for a survey of the most recent methods in the field.

1.2 Difference from existing work

Our contributions are as follows

- The gaussian mixtures used in [28] are sensitive to initialisation and parameters such as the number of peaks, while assuming independence of each activation value. They then use the hamming distance to support explainabilty of their predictions
-

2 Methods

2.1 Class-Wise Frequencies of Significant Neuron-Weight Interactions

As an additional approach, we decided to study the interactions in a neural network at a more granular level. Instead of just looking at the final value of the neuron, we decided to look at each individual weight being multiplied into each neuron. In this case, what we did is described below.

The final layer of the neural network operates as follows. Given an activation vector $x \in \mathbb{R}^{n_{in} \times 1}$ and a weight matrix $W \in \mathbb{R}^{n_{out} \times n_{in}}$

$$\begin{bmatrix} w_{1,1} & \dots & w_{1,n_{in}} \\ \vdots & \ddots & \vdots \\ w_{n_{out},1} & \dots & w_{n_{out},n_{in}} \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_{n_{in}} \end{bmatrix} = \begin{bmatrix} w_{1,1}x_1 + w_{1,2}x_2 + \dots + w_{1,n_{in}}x_{n_{in}} \\ \vdots \\ w_{n_{out},1}x_1 + w_{n_{out},2}x_2 + \dots + w_{n_{out},n_{in}}x_{n_{in}} \end{bmatrix}$$

This would not allow us to study the individual activation-weight pairs, because they would all be summed up. A vector operation that computes the sum over the rows of a matrix is postmultiplication by the all 1s vector, so by factoring out this vector we can get the required information. Factoring out the all 1s vector from the neurons in the penultimate layer, we get an diagonal matrix with the neurons activations as entries

$$\begin{bmatrix} w_{1,1} & \dots & w_{1,n_{in}} \\ \vdots & \ddots & \vdots \\ w_{n_{out},1} & \dots & w_{n_{out},n_{in}} \end{bmatrix} \begin{bmatrix} x_1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & x_{n_{in}} \end{bmatrix} = \begin{bmatrix} w_{1,1}x_1 & \dots & w_{1,n_{in}}x_1 \\ \vdots & \ddots & \vdots \\ w_{n_{out},1}x_{n_{in}} & \dots & w_{n_{out},n_{in}}x_{n_{in}} \end{bmatrix}$$

To this, we concatenated the bias vector as an additional neuron, giving us a matrix in $\mathbb{R}^{n_{out} \times (n_{in}+1)}$ The definition of significance used was

$$S(x_{ij}) = 1 \text{ if } |x_{ij}| > \frac{|\sum_j x_{ij}|}{n_{in}}$$

where n_{in} is the number of input features, i.e. number of columns in the matrix. The justification behind this is the idea that if an activation has an above average contribution to the final activation, the neuron-weight interaction is significant. As eariler, we hypothesise that the number of significant interactions between neurons and weights are sparse, and that the sparsity increases with depth.

2.2 KL Divergence Between Classes

Building on the same experiment, we wanted to check both the entropies in between different classes over the activation distribution for the same layer. We also measured the same entropies at training and testing time, as well as an in and out of distribution dataset. The in-distribution dataset was the validation set for imagenet as used eariler, and the out of distribution dataset was the imagenet-r dataset created by Hendrycks et. al.[8]. This is a collection of stylised versions of the standard classes of imagenet such as caricatures. This dataset has 200 classes compared to the original 1000 from imagenet, and to ensure the comparison was meaningful we sampled 50 classes from the dataset uniformly at random and computed the pairwise KL divergences between pairs of classes from the imagenet validation set, between pairs of classes from the imagenet-r set and between the same classes from imagenet and imagenet-r. Along similar lines, we also measured the entropy of the distributions. Analagous to regression, the distance within the clusters (entropy) should decrease, while the distance between the clusters (KL divergence) should increase. The entropy is given by the formula

$$H(X) = -\sum p(x)\log(p(x))$$

Where $p(x)$ is the probability that the random variable X takes on the value X. Intuitively, it is a measure of 'disorder' in the random variable. The KL divergence is the relative entropy between 2 distributions, given by the formula

$$D_{KL}(P||Q) = \sum P(x)\log(\frac{P(x)}{Q(x)})$$

This can be thought of as a measure of how well P approximates Q. It is not a true metric, because it doesn't satisfy triangle inequality or symmetry. Our computations make a strong assumption of **independence** of the significance of activations, i.e. that any activation will be significant in it's own row is independent of the other events being significant.

2.3 Modelling Activations using a Multivariate Gaussian

The matrix of neuron-weight products for each image can flattened

$$\begin{bmatrix} w_{1,1}x_1 & \dots & w_{1,n_{in}}x_1 \\ \vdots & \ddots & \vdots \\ w_{n_{out},1}x_{n_{in}} & \dots & w_{n_{out},n_{in}}x_{n_{in}} \end{bmatrix} \rightarrow \begin{bmatrix} w_{1,1}x_1 \\ \vdots \\ w_{n_{out},n_{in}}x_{n_{in}} \end{bmatrix}$$

The following estimators can be used for the mean and variance of the distribution. Note that this assumes the distribution is gaussian, because it is being modelled entirely using just the mean and variance. Given images $x_1, \dots, x_n$ for a class c, it's mean and covariance can be estimated as

$$\vec{\mu_c} = \frac{1}{n}\sum_{i=1}^{n}\vec{x}_i$$

$$\Sigma_c = \frac{1}{n}\sum_{i=1}^{n}(\vec{x_i} - \vec{\mu_c})(\vec{x_i} - \vec{\mu_c})^T$$

To avoid issues with numerical stability, the off-diagonal entries are set to zero, effectively assuming independence of the neuron activation values. Then, the mahalanobis distance is calculated to estimate the distance between classes at the final layer.

$$d_{\text{mahalanobis}}(c_1, c_2) = \sqrt{((\vec{\mu_{c_1}} - \vec{\mu_{c_2}})^T \Sigma_{c_1} (\vec{\mu_{c_1}} - \vec{\mu_{c_2}}))}$$

2.4 Memorisation and Significance

We hypothesise that a network which has memorised it's training data would have significantly different properties in terms of sparsity and the separation of classes. To this end, we replicate the experiment specified in [30] and train a modified version of AlexNet on CIFAR10 until it reaches zero error. The details of the training stay the same and will be provided in Appendix A.

2.5 Adversarial Attacks

The above gives us a distribution on the activations for a given class. We compared the probabilities conditioned on the distribution of the class of the validation images subject when subjected to a PGD attack [16]. The parameters of the attack are given in Appendix B.

2.6 Manifold Entaglement

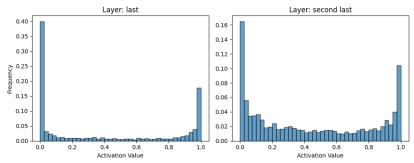
In the final hidden layer, the low-dimensional manifolds for different classes should be linearly separable. This is because the remaining transformations the network will apply are an affine hidden layer computation and a linear softmax computation. We estimate the dimensionality of the manifolds spanned by each class by inspecting the neurons for this layer, combining them into a matrix and finding it's rank. We further estimate the degree of overlap between the manifolds by combining the matrices for different classes and then applying the principle of inclusion and exclusion. Networks that have not overfit their training data should have manifolds with little overlap, while those which have should have entagled class manifolds. We additionally expect the manifolds to have smaller dimensionality for networks with better embeddings.

3 Results

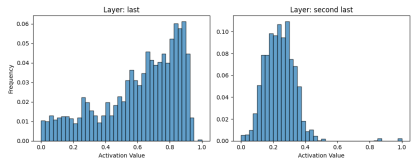
We examined the distributions for AlexNet, VGG, ResNet and InceptionV3. The experiments were run using PyTorch, and the weights for the pretrained versions of the networks were also taken from there. The datasets used were the validation sets for Imagenet [24], and CIFAR10 [12]

3.1 Sparsity of Activations

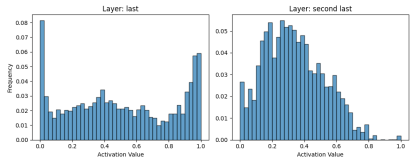
The following plots have the number of times a neuron activated over the entire validation set divided by the length of the validation set on the x axis, and the y-axis is the number of neurons that activate in that 'bin', divided by the total number of neurons. The results indicate that training encourages sparsity quite strongly, with the proportions of neurons that are always on dropping from 0.2 and 0.1 to 0.05 and < 0.01 on the last and second last layers respectively from the untrained model to the best model. The network fitting random labels has an extremely sparse penultimate layer, with the best network having some density to it and the overtrained network resembling a mid-point in between the random labelled and the best network.



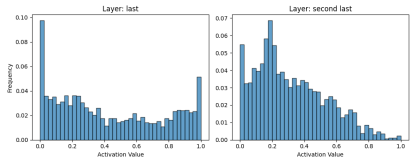
(a) Untrained



(b) Random Labels

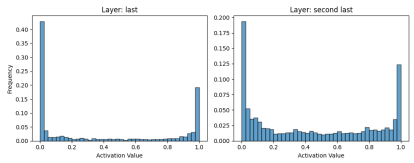


(c) Overtrained

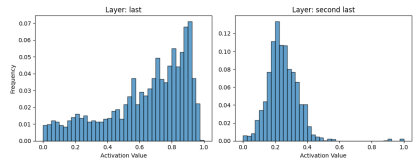


(d) Best

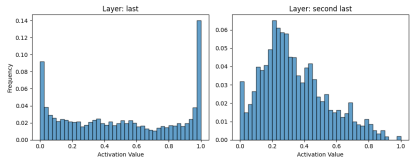
Figure 1: Activation Proportions from the class Airplane on the last and second last layers on different versions of the Modified AlexNet



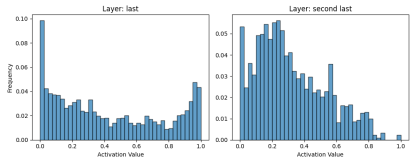
(a) Untrained



(b) Random Labels



(c) Overtrained



(d) Best

Figure 2: Activation Proportions from the class Frog on the last and second last layers on different versions of the Modified AlexNet

The proportion histograms for the last 3 layers of alexnet on imagenet and imagenet-r are in figure 3. For AlexNet, we can see that the randomly initialised network looks the same in terms of activation proportions for all the layers across both datasets. We once again see that training encourages sparsity in the network, even on the out of distribution set. The in-distribution dataset is sparser on the final layer and the penultimate layers. In fact, there is a significant increase in the proportion of stray activations on the out of distribution dataset, suggesting that sparsity may be a characteristic of learning.

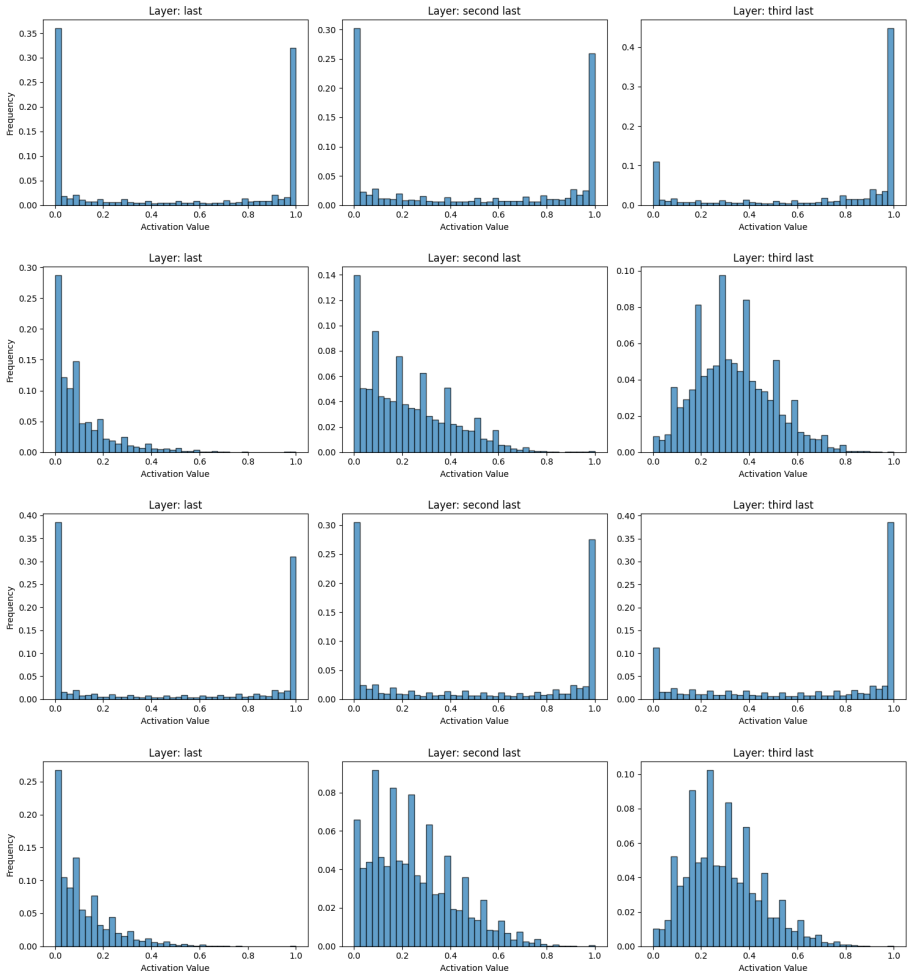


Figure 3: Frequencies of the significant neuron-weight activation proportions for the networks: untrained on imagenet, trained on imagenet, untrained on imagenet-r and trained on imagenet-r from top to bottom

3.2 Divergences of Neuron Weight Interactions

Figures 4 and 5 are the results for the KL divergences of alexnet after correcting for the dimensionality of the space. Each row represents the KL divergence from a class, while a column represents the KL divergences to a class. The values on the diagonals for KL divergence would have been 0 and we plot the entropy of the distribution for that class instead. Between initialisation and the end of training on the in-distribution dataset, we see that the separation of classes increases greatly, while the entropy of the class distributions decreases simultneously. This decrease is much more noticable at the deeper layers of the

network, and also extremely telling between the networks for the in-distribution classes and out of distribution classes. The OOD classes become more separated from each other but their entropies see no decrease after training is complete, even seeing significant increases in some cases.

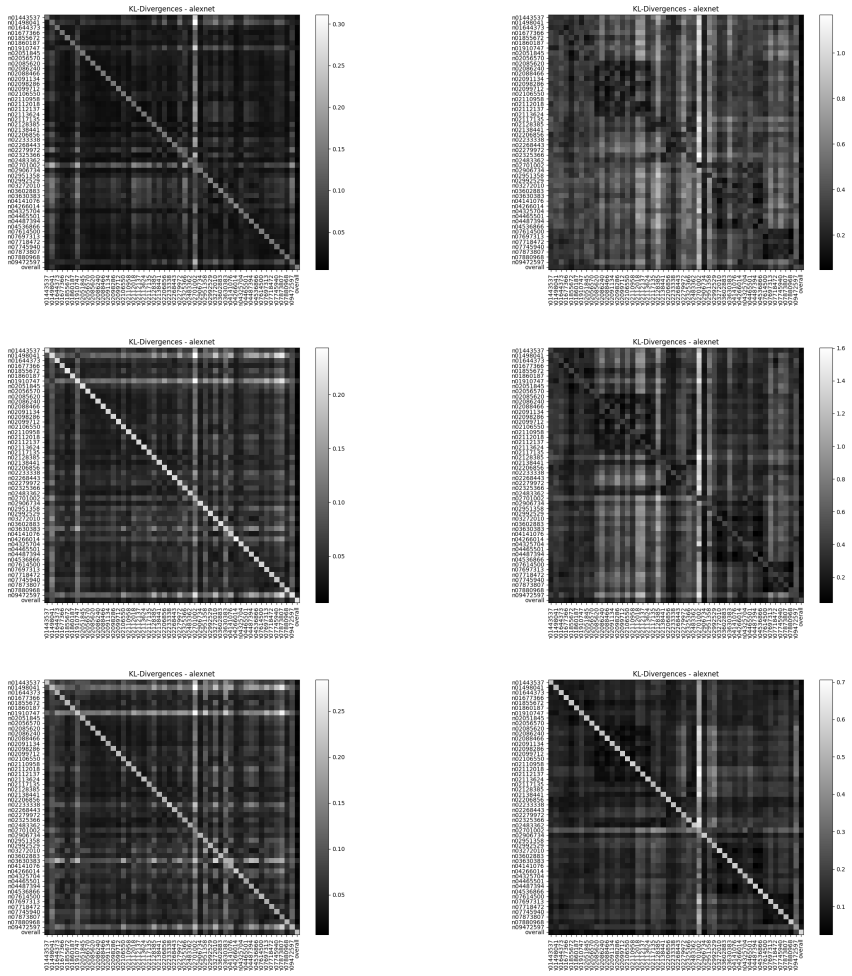


Figure 4: KL divergences between different classes for the untrained (left) and trained (right) classes of alexnet on imagenet. From top to bottom, these are the values for the distributions for the last, second last and third last layers

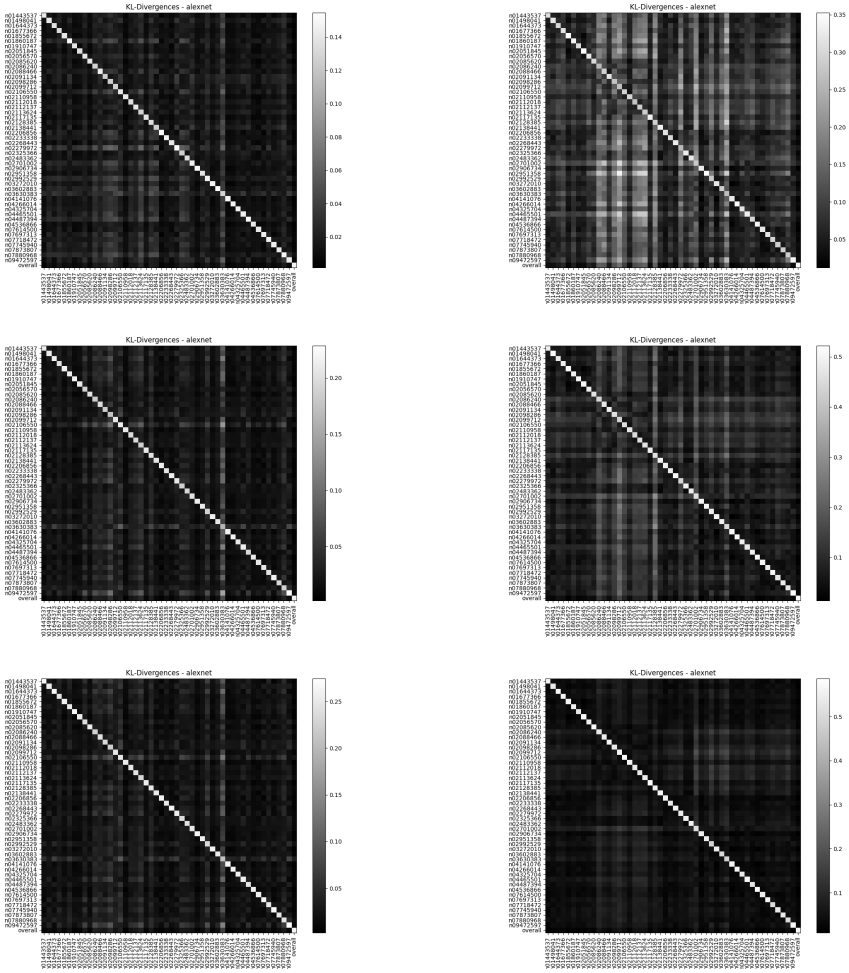


Figure 5: KL divergences between different classes for the untrained (left) and trained (right) classes of alexnet on imagenet-r. From top to bottom, these are the values for the distributions for the last, second last and third last layers

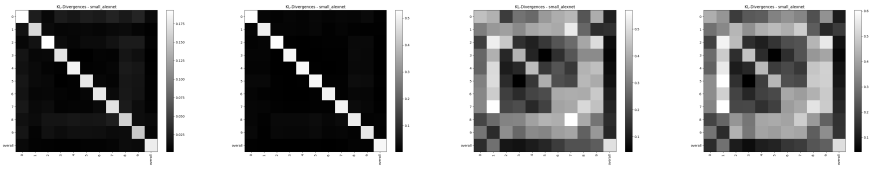
Figures 6 and 7 show the divergences between classes and their OOD counterparts before and after training. We observe that the separation of the same class inside and outside the distribution decreases. The distributions becoming more similar might suggest that the model is in fact learning some characteristics common to across the training distribution. . Additionally, the OOD distribution has a larger divergence from the ID distribution than the other way around. This would suggest that the OOD distribution is more chaotic than its ID counterpart. This is another difference that is more pronounced at the deeper layers, since as we saw earlier the entropy of the in-distribution

activations reduces with depth.

Figure 6: KL divergences from the in-distribution classes to the out of distribution classes untrained (left) and trained (right). From top to bottom, these are for the final, second last layer and third last layers

Figure 7: KL divergences from the out-of-distribution classes to the in distribution classes untrained (left) and trained (right). From top to bottom, these are for the final, second last layer and third last layers

The KL divergence heatmaps for the CIFAR10 networks are in Figure 8. The entropy is larger than the separation at initialisation, but training on random labels greatly increases the entropy of the distribution with a small difference in the inter class separation. For the best and overtrained networks, we see almost identical patterns of separation between classes, with the magnitude being smaller for the network. The decrease in relative separation might suggest that the network is 'forgetting' the relationships between classes and making the distributions more similar.



(a) Untrained (b) Random Labels (c) Overtrained (d) Best

Figure 8: KL divergence heatmaps for the modified Alexnet models on CIFAR10

3.3 Adversarial Attacks and Probability Distributions

Presented below are the results for just the modified version of AlexNet on CIFAR10, because running this would not be meaningful on an untrained network or an OOD dataset. The results for the version of the network on random labels compared to the version trained to convergence normally are presented below.

We see that on the network with random labels, there is no consistent effect that the attack seems to have. Some classes see a sharp increase in the probabilities under the class distribution, while some see a decrease. On the trained network, however, we see a more consistent decrease in likelihood under the class distribution from the clean samples to the adversarial samples.

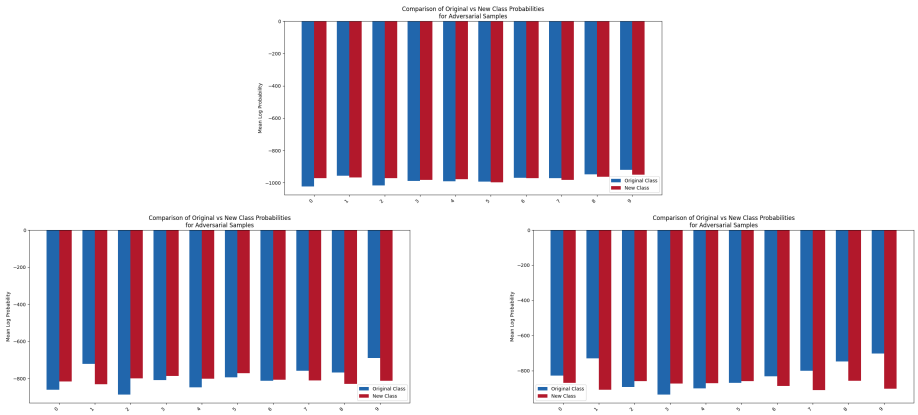


Figure 9: Adversarial Probability Comparison for different classes for the random network (top), best network (left) and the overtrained network (right)

3.4 Manifold Entanglement

The difference between the manifolds for both networks is quite stark. The random network not only has significant overlap between classes, the manifolds also have much higher rank. The clean network doesn't see any overlap at all, and has lower rank manifolds. Additionally, we see that the overtrained manifold has several classes actually increase in rank. This lines up with the hypothesis of the 'optimal' solution being a substructure with lower rank.

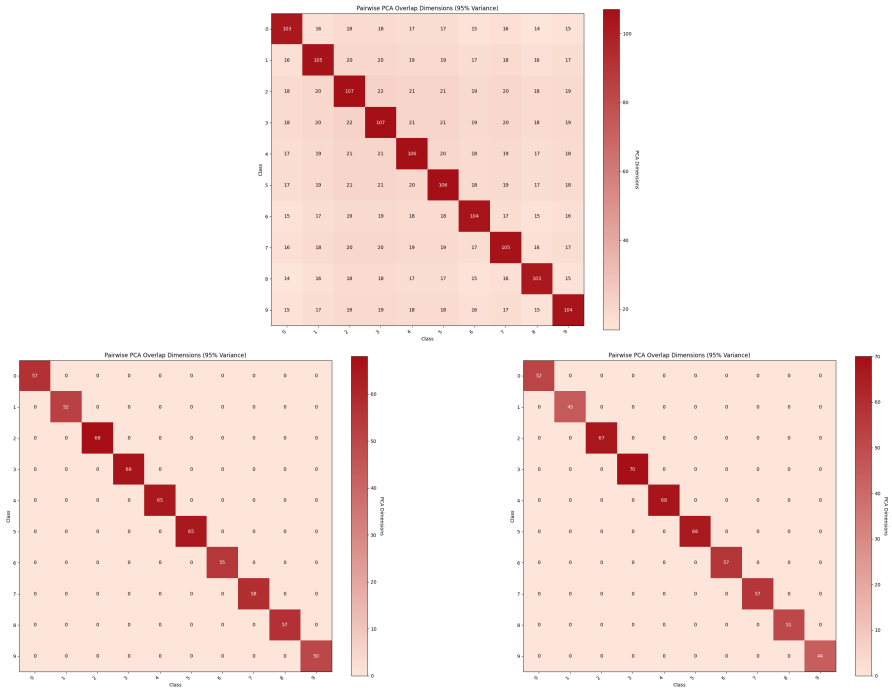


Figure 10: Pairwise Entaglements for different classes for the random network (left) and the clean network (right)

4 Conclusions and Future Work

The above results present a new way of looking at and analysing a trained neural network. We analyse the distributions of class activations and the effect of training on their evolution. We also analyse the change in probability under an adversarial attack and the entanglement of class manifolds under different conditions. We believe that as an evaluation framework, this is more robust to overfitting by inspecting the similarities at higher layers as well as lower layers. The linear manifold model using PCA makes particularly strong assumptions such as gaussian data distribution, which the KL divergence model does not. An immediate advantage of our proposed framework is a very noticable change on an overtrained network, where the PCA manifolds see a subtle increase in dimension, our framework sees a noticable decrease in the separation of the class distributions. Our framework additionally shows different behaviour on overfitting and registers different behaviours for in-distribution data and OOD data. We believe this framework can be further extended to build a tool that can enhance robustness to both adversarial and tail inputs as well as detect poor generalisation.

Future directions we are currently considering are

- Defining significant activations for other layer types, such as convolutional or attention layers
- Characterising dataset complexity/conditioning in terms of the KL divergence and entropy of their activations on random networks
- Bounding the maximum possible KL divergence between classes on a single dataset and architecture
- Finding the sample complexity of increasing the KL divergence to within a $(1 - \epsilon)$ factor of the maximum

References

[1] Sanjeev Arora, Nadav Cohen, and Elad Hazan. On the optimization of deep networks: Implicit acceleration by overparameterization. In Jennifer Dy and Andreas Krause, editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 244–253. PMLR, 10–15 Jul 2018.

[2] Sanjeev Arora, Nadav Cohen, Wei Hu, and Yuping Luo. Implicit regularization in deep matrix factorization, 2019.

[3] Devansh Arpit, Stanisław Jastrzębski, Nicolas Ballas, David Krueger, Emmanuel Bengio, Maxinder S. Kanwal, Tegan Maharaj, Asja Fischer, Aaron Courville, Yoshua Bengio, and Simon Lacoste-Julien. A closer look at memorization in deep networks, 2017.

[4] Alex Bäuerle, Daniel Jönsson, and Timo Ropinski. Neural activation patterns (naps): Visual explainability of learned concepts, 2022.

[5] Charles Fefferman, Sanjoy Mitter, and Hariharan Narayanan. Testing the manifold hypothesis, 2013.

[6] Daniel Gissin, Shai Shalev-Shwartz, and Amit Daniely. The implicit bias of depth: How incremental learning drives generalization, 2019.

[7] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples, 2015.

[8] Dan Hendrycks, Steven Basart, Norman Mu, Saurav Kadavath, Frank Wang, Evan Dorundo, Rahul Desai, Tyler Zhu, Samyak Parajuli, Mike Guo, Dawn Song, Jacob Steinhardt, and Justin Gilmer. The many faces of robustness: A critical analysis of out-of-distribution generalization, 2021.

[9] Dan Hendrycks and Kevin Gimpel. A baseline for detecting misclassified and out-of-distribution examples in neural networks, 2018.

[10] Dan Hendrycks, Mantas Mazeika, and Thomas Dietterich. Deep anomaly detection with outlier exposure, 2019.

[11] Fred Hohman, Haekyu Park, Caleb Robinson, and Duen Horng Chau. Summit: Scaling deep learning interpretability by visualizing activation and attribution summarizations, 2019.

[12] Alex Krizhevsky. Learning multiple layers of features from tiny images, 2009.

[13] Kimin Lee, Kibok Lee, Honglak Lee, and Jinwoo Shin. A simple unified framework for detecting out-of-distribution samples and adversarial attacks, 2018.

[14] Mingchen Li, Mahdi Soltanolkotabi, and Samet Oymak. Gradient descent with early stopping is provably robust to label noise for overparameterized neural networks. In Silvia Chiappa and Roberto Calandra, editors, *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*, volume 108 of *Proceedings of Machine Learning Research*, pages 4313–4324. PMLR, 26–28 Aug 2020.

[15] Shiyu Liang, Yixuan Li, and R. Srikant. Enhancing the reliability of out-of-distribution image detection in neural networks, 2020.

[16] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks, 2019.

[17] Marina Meilă and Hanyu Zhang. Manifold learning: what, how, and why, 2023.

[18] Hariharan Narayanan and Sanjoy Mitter. Sample complexity of testing the manifold hypothesis. In J. Lafferty, C. Williams, J. Shawe-Taylor, R. Zemel, and A. Culotta, editors, *Advances in Neural Information Processing Systems*, volume 23. Curran Associates, Inc., 2010.

[19] Behnam Neyshabur. Implicit regularization in deep learning, 2017.

[20] Behnam Neyshabur, Ryota Tomioka, and Nathan Srebro. In search of the real inductive bias: On the role of implicit regularization in deep learning, 2015.

[21] Bartlomiej Olber, Krystian Radlak, Adam Popowicz, Michal Szczepankiewicz, and Krystian Chachuła. Detection of out-of-distribution samples using binary neuron activation patterns, 2023.

[22] Colin Paterson, Radu Calinescu, and Chiara Picardi. Detection and mitigation of rare subclasses in deep neural network classifiers, 2021.

[23] Noam Razin and Nadav Cohen. Implicit regularization in deep learning may not be explainable by norms, 2020.

[24] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. Imagenet large scale visual recognition challenge, 2015.

[25] Chandramouli Shama Sastry and Sageev Oore. Detecting out-of-distribution examples with in-distribution examples and gram matrices, 2020.

[26] Ali Shafahi, W. Ronny Huang, Christoph Studer, Soheil Feizi, and Tom Goldstein. Are adversarial examples inevitable?, 2020.

[27] Adi Shamir, Odelia Melamed, and Oriel BenShmuel. The dimpled manifold model of adversarial examples in machine learning, 2022.

[28] Martin Stano, Wanda Benesova, and Lukas Samuel Martak. Explaining predictions of deep neural classifier via activation analysis, 2020.

[29] Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. Intriguing properties of neural networks, 2014.

[30] Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization, 2017.

A Details for Modified Alexnet

The modified version of Alexnet is adapted to the CIFAR10 dataset, which is much smaller than the Imagenet dataset that Alexnet was built for. The convolutional block here is a 5×5 convolution, followed by a 3×3 max pool and a local response normalisation layer with size 5, $\alpha = 0.0001$, $\beta = 0.75$ and $k = 2$. Two convolutional blocks are followed by a two fully-connected hidden layers with 384 and 192 neurons, with the output having 10 neurons. The output is softmax activated, while the other layers are ReLU activated.

The optimizer used is SGD, with the initial learning rate set to 0.01, decaying by 0.95 after every epoch. No explicit regularisation such as dropout or weight decay are used. Each 32×32 image is cropped to a 28×28 resolution, with each image *independently* standardised to have zero mean and unit standard deviation.

B Appendix B: PGD Attack Details

The Projected Gradient Descent attacks were carried out in 50 iterations, with a step size of 0.001 and a radius of 0.03. The PCA was thresholded until it explained 95% of the variance from the activations.